

7_Type_casting in python

September 15, 2026

```
[ ]: # External Data type casting in Python
x = 5 # integer
y = 2.5 # float
z = "10" # string
print("Original data types:")
print("x:", type(x))
print("y:", type(y))
print("z:", type(z))
print("\nCasting data types:")
print(x + y + int(z)) # casting string to integer
print(type(x + y + int(z)))
```

Original data types:

x: <class 'int'>
y: <class 'float'>
z: <class 'str'>

Casting data types:

17.5
<class 'float'>

```
[2]: # Internal Type Conversion
a = 10 # integer
b = 3.5 # float
result = a + b # automatic type conversion to float
print("\nInternal Type Conversion:")
print("a:", type(a))
print("b:", type(b))
print("Result of a + b:", result)
```

Internal Type Conversion:

a: <class 'int'>
b: <class 'float'>
Result of a + b: 13.5

```
[5]: # convert the values which are compatible only
print(int('10a')) # This will raise a ValueError because '10a' cannot be
↳ converted to an integer
```

```

-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_53555/3922154625.py in <cell line: 0>()
      1 # convert the values which are compatible only
----> 2 print(int('10a')) # This will raise a ValueError because '10a' cannot
      ↳ be converted to an integer

ValueError: invalid literal for int() with base 10: '10a'

```

```

[7]: emp_name="Hanumantha Rao Allada"
print(tuple(emp_name)) # This will raise a TypeError because a string cannot
      ↳ be directly converted to a tuple
print(list(emp_name)) # This will raise a TypeError because a string cannot be
      ↳ directly converted to a list
print(set(emp_name)) # This will raise a TypeError because a string cannot be
      ↳ directly converted to a set

```

```

('H', 'a', 'n', 'u', 'm', 'a', 'n', 't', 'h', 'a', ' ', 'R', 'a', 'o', ' ', 'A',
'l', 'l', 'a', 'd', 'a')
['H', 'a', 'n', 'u', 'm', 'a', 'n', 't', 'h', 'a', ' ', 'R', 'a', 'o', ' ', 'A',
'l', 'l', 'a', 'd', 'a']
{'l', 'o', 'n', 'H', ' ', 'u', 'R', 'a', 'd', 'm', 'h', 't', 'A'}

```

```

[9]: # list to tuple conversion
my_list = [1, 2, 3, 4, 5]
my_tuple = tuple(my_list)
print("Original list:", my_list)
print("Converted tuple:", my_tuple)

```

```

Original list: [1, 2, 3, 4, 5]
Converted tuple: (1, 2, 3, 4, 5)

```

```

[10]: #tuple to list conversion
my_tuple = (1, 2, 3, 4, 5)
my_list = list(my_tuple)
print("Original tuple:", my_tuple)
print("Converted list:", my_list)

```

```

Original tuple: (1, 2, 3, 4, 5)
Converted list: [1, 2, 3, 4, 5]

```

```

[3]: a=1
b=2
c=complex(a,b)
print("Complex number:", c)
print("Real part:", c.real)
print(type(c.real))

```

```
print("Imaginary part:", c.imag)
print(type(c.imag))
print("Type of complex number:", type(c))
```

Complex number: (1+2j)

Real part: 1.0

<class 'float'>

Imaginary part: 2.0

<class 'float'>

Type of complex number: <class 'complex'>

```
[ ]: # Creating a dict using zip from nested tuples
tup = ((1, 'a'), (2, 'b'), (3, 'c'), (4, 'd'), (5, 'e'))
print(dict(tup))
```

{1: 'a', 2: 'b', 3: 'c', 4: 'd', 5: 'e'}

```
[8]: value=1
print(bool(value))
value1=0
print(bool(value1))
```

True

False